

TAREA — TRABAJO DE PLANEAMIENTO

Punto de Partida de tu Proyecto

Elige tu proyecto y describe cómo va hasta la Sesión 9

Desarrollo de Aplicaciones Básico

Alumno: Stephano Camarena Villa

Proyecto: Zoapex

Fecha: 11 de junio de 2026

1. Elige tu proyecto

Mi proyecto se llama Zoapex y es una aplicación de escritorio de e-commerce para una tienda de mascotas. El prototipo se centra en mostrar el catálogo de productos y en registrar las ventas de esos productos, pero el diseño queda preparado para crecer hacia un sistema más completo. Elegí este rubro porque me permite cumplir de forma clara las tres condiciones que se aplicarán más adelante en el curso:

- **Mantenimientos CRUD:** gestiono el alta, edición y eliminación de productos y de categorías (y, en una fase posterior, de clientes).
- **Operación maestro-detalle transaccional:** el registro de un pedido, con una cabecera y varias líneas de detalle, que debe grabarse “todo o nada”. Mi pareja es Pedido → DetallePedido (en la base de datos, order → order_detail).
- **Consultas y reportes:** el catálogo ya combina las tablas producto y categoría, y a futuro tendré un reporte de ventas que cruza pedido, detalle, producto y cliente, pensado para poder exportarse a Excel.

2. Describe de qué trata tu proyecto

Zoapex es un sistema para una tienda de mascotas que ordena su catálogo y registra sus ventas. Lo usan dos perfiles: el administrador, que mantiene los productos y las categorías (precios, stock, descripciones, imagen), y el usuario final o vendedor, que navega el catálogo, arma un carrito y concreta la compra. Toda la información vive en una base de datos PostgreSQL alojada en Supabase (en la nube), y la aplicación de escritorio se conecta a ella para leer y grabar datos.

La operación más importante es el registro de la venta y ocurre así, paso a paso: el usuario abre el catálogo y ve los productos traídos desde Supabase; va agregando productos al carrito indicando la cantidad de cada uno; al confirmar, el sistema valida que haya stock suficiente, calcula el subtotal, le suma el IGV del 18 % y obtiene el total; finalmente graba el pedido junto con todas sus líneas de detalle de forma transaccional y descuenta el stock de cada producto. Si cualquier paso falla, no se guarda nada, manteniendo la base de datos consistente. Por eso mi pareja maestro-detalle es Pedido → DetallePedido (order → order_detail).

3. Cómo va mi proyecto hasta la Sesión 9

Apoyándome en lo visto en clase (Windows Forms y MDI, capas con CRUD en ADO.NET, reportes a Excel y la operación maestro-detalle transaccional), tengo definido el plano de mi sistema de la siguiente manera:

- **Arquitectura por capas:** el proyecto se organiza en 4 proyectos independientes. Zoapex.Entities contiene las clases POCO (es el núcleo y no referencia a nadie); Zoapex.DataAccess maneja la conexión a Supabase con el driver Npgsql y tiene una clase DAL por entidad; Zoapex.Business concentra todas las validaciones y el cálculo de totales; y Zoapex.Presentation es la interfaz en WPF. Las referencias van en una sola dirección: Presentation → Business → DataAccess → Entities, de modo que la interfaz nunca habla directo con la base de datos.
- **Tablas principales:** mis tablas maestras son category, product y customer. La pareja cabecera-detalle es order (cabecera del pedido) con order_detail (las líneas), enlazadas para guardar la venta completa.
- **Módulos previstos:** para el administrador, el CRUD de productos y de categorías y, más adelante, el reporte de ventas exportable a Excel. Para el usuario final, el catálogo visual, el carrito de compras y el registro de la venta con descuento automático de stock.
- **Estado actual:** lo más avanzado es el diseño de la arquitectura y la base de datos: ya tengo definidas las 5 tablas, 7 funciones de PostgreSQL (equivalentes a stored procedures) y los datos de prueba. Me falta construir el código de cada capa y las ventanas en WPF, y dejo para fases siguientes el login con roles, la gestión completa de clientes y los dashboards.

Como elementos diferenciadores, mi base de datos está en la nube (Supabase / PostgreSQL) en lugar de un SQL Server local, uso WPF en vez de Windows Forms y todo el código (clases, métodos y variables) está en inglés, dejando los comentarios en español.

4. ¿En qué tecnología hago el CRUD de administración?

Para mi proyecto elegí el esquema Form + Web. La parte administrativa (el CRUD de productos y categorías) la desarrollaré en escritorio con WPF, que en mi caso reemplaza a Windows Forms como diferenciador; y la cara pública que usará el cliente (el catálogo y la tienda) la llevaré a la web con ASP.NET Core en una fase posterior. Así, el administrador trabaja cómodo desde una app de escritorio y el cliente final puede comprar desde el navegador.

Respeto la regla de oro en todo momento: la interfaz nunca habla directo con la base de datos. Las capas Business, DataAccess y Entities se reutilizan exactamente igual en escritorio o en web, y lo único que cambia es la presentación. Por ahora esto es solo el plano: el CRUD web real lo construiré más adelante con Entity Framework y LINQ, tal como veremos en las Sesiones 12-13.

Parte 4 - Aplicación de la Sesión 12

Esta sección documenta la implementación de Entity Framework Core, LINQ, procedimientos almacenados (funciones PostgreSQL) y AJAX en la parte web del proyecto Zoapex, continuando el documento de planeamiento entregado en la PA3.

Decisión y justificación general

Para Zoapex mantuve la Ruta A (Escritorio + Web) definida en la PA3. El CRUD de productos, categorías y la operación maestro-detalle transaccional (Pedido → DetallePedido) se gestionan desde la aplicación de escritorio WPF con arquitectura en N capas y ADO.NET (Npgsql). La parte web (ASP.NET Core Razor Pages) se conecta a la misma base de datos en Supabase y permite al cliente ver el catálogo, iniciar sesión, armar un carrito y confirmar su compra.

En la web utilizo EF Core como ORM principal para mapear las tablas de PostgreSQL, LINQ para las consultas del catálogo e historial de pedidos del cliente, y la función `fn_register_order` como procedimiento almacenado para registrar la venta de forma transaccional (cabecera + detalle + descuento de stock). El AJAX lo apliqué en el catálogo y en el carrito para filtrar productos, ver el detalle y actualizar totales sin recargar la página.

Respeto la regla de oro: la vista nunca consulta la base de datos directamente. Las páginas Razor llaman a repositorios en la capa Data (CatalogRepository, OrderRepository, CustomerRepository), y estos usan EF Core, LINQ o la invocación al procedimiento almacenado.

A. Entity Framework Core - DbContext y modelo de datos

Justificación: uso EF Core para mapear las tablas de Supabase (category, product, customer, order, order_detail) como entidades C# y centralizar las relaciones entre ellas. Esto me permite reutilizar el mismo contexto en todos los repositorios de la capa web.

Archivo: Zoapex.Web/Data/ZoapexDbContext.cs

► Zoapex.Web/Data/ZoapexDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using Zoapex.Web.Data.Models;

namespace Zoapex.Web.Data;

public class ZoapexDbContext(DbContextOptions<ZoapexDbContext> options) : DbContext(options)
{
    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Product> Products => Set<Product>();
    public DbSet<Customer> Customers => Set<Customer>();
    public DbSet<OrderHeader> Orders => Set<OrderHeader>();
    public DbSet<OrderDetail> OrderDetails => Set<OrderDetail>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Product>()
            .HasOne(p => p.Category)
            .WithMany(c => c.Products)
            .HasForeignKey(p => p.CategoryId);

        modelBuilder.Entity<OrderHeader>()
            .HasOne(o => o.Customer)
            .WithMany(c => c.Orders)
            .HasForeignKey(o => o.CustomerId);

        modelBuilder.Entity<OrderDetail>()
            .HasOne(d => d.Order)
            .WithMany(o => o.Details)
            .HasForeignKey(d => d.OrderId);

        modelBuilder.Entity<OrderDetail>()
            .HasOne(d => d.Product)
            .WithMany()
            .HasForeignKey(d => d.ProductId);
    }
}
```

Explicación: el DbContext define los DbSet de cada tabla y configura las relaciones Product → Category, Order → Customer y OrderDetail → Order/Product. Las entidades usan atributos [Table] y [Column] para mapear los nombres en snake_case de PostgreSQL.

B. LINQ - catálogo e historial de pedidos

Justificación: para listados y filtros frecuentes (catálogo por categoría, búsqueda por nombre/código y historial del cliente) uso LINQ to Entities porque la consulta queda legible en C# y el compilador valida los nombres de propiedades.

Archivo: *Zoapex.Web/Data/CatalogRepository.cs* — filtro del catálogo

► Zoapex.Web/Data/CatalogRepository.cs · GetCatalogAsync()

```
// LINQ: listado del catálogo con nombre de categoría (JOIN implícito)
public async Task<List<ProductCardDto>> GetCatalogAsync(string? search, int? categoryId)
{
    var query = ctx.Products
        .AsNoTracking()
        .Include(p => p.Category)
        .Where(p => p.Status == 1);

    if (categoryId is > 0)
        query = query.Where(p => p.CategoryId == categoryId);

    if (!string.IsNullOrEmpty(search))
    {
        var term = search.Trim().ToLower();
        query = query.Where(p =>
            p.Name.ToLower().Contains(term) ||
            p.Code.ToLower().Contains(term) ||
            (p.Category != null && p.Category.Name.ToLower().Contains(term)));
    }

    return await query
        .OrderBy(p => p.Code)
        .Select(p => new ProductCardDto(
            p.ProductId,
            p.Code,
            p.Name,
            p.Description ?? string.Empty,
            p.Price,
            p.Stock,
            p.Category!.Name))
        .ToListAsync();
}
```

Explicación: esta consulta LINQ filtra productos activos por categoría y texto de búsqueda, hace un JOIN implícito con Category mediante Include y proyecta el resultado a un DTO para la vista.

► Zoapex.Web/Data/OrderRepository.cs · GetCustomerOrdersAsync()

```
// LINQ: historial del cliente autenticado
public async Task<List<OrderHistoryDto>> GetCustomerOrdersAsync(int customerId, int take = 20)
{
    return await ctx.Orders
        .AsNoTracking()
        .Include(o => o.Details)
        .ThenInclude(d => d.Product)
        .Where(o => o.Status == 1 && o.CustomerId == customerId)
        .OrderByDescending(o => o.OrderDate)
        .Take(take)
        .Select(o => new OrderHistoryDto(
            o.OrderId,
            o.Code,
            o.OrderDate,
            o.Subtotal,
            o.Tax,
            o.Total,
            o.Details
                .Select(d => new OrderDetailLineDto(
                    d.Product.Name,
                    d.Quantity,
                    d.UnitPrice,
                    d.Subtotal))
                .ToList()))
        .ToListAsync();
}
```

Explicación: trae los pedidos del cliente autenticado con sus líneas de detalle y el nombre del producto, ordenados del más reciente al más antiguo. Se usa en la página Mis pedidos.

C. Procedimiento almacenado - registro transaccional del pedido

Justificación: el registro de la venta (cabecera + detalle + cálculo de IGV + descuento de stock) involucra varias operaciones que deben ejecutarse 'todo o nada'. Por eso delego esa lógica a la función `fn_register_order` en PostgreSQL (equivalente a un stored procedure en SQL Server), reutilizando la misma función que ya usa la aplicación de escritorio WPF.

1) Función creada en Supabase (`zoapex-db/sql/zoapex_database.sql`)

► zoapex-db/sql/zoapex_database.sql · fn_register_order()

```
-- fn_register_order: Registra pedido + detalle en transacción y descuenta stock
-- p_customer_id puede ser NULL para ventas sin cliente registrado (mostrador)
CREATE OR REPLACE FUNCTION fn_register_order(
    p_customer_id INT,
    p_details JSONB -- [{product_id, quantity, unit_price}]
)
RETURNS INT AS $$
DECLARE
    v_order_id INT;
    v_next_id INT;
    v_code VARCHAR(12);
    v_subtotal NUMERIC(10,2) := 0;
    v_tax NUMERIC(10,2);
    v_total NUMERIC(10,2);
    v_detail JSONB;
    v_qty INT;
    v_price NUMERIC(10,2);
    v_prod_id INT;
    v_line_subtotal NUMERIC(10,2);
BEGIN
    -- Generar código del pedido
    SELECT COALESCE(MAX(order_id), 0) + 1 INTO v_next_id FROM "order";
    v_code := 'ORD' || LPAD(v_next_id::TEXT, 3, '0');

    -- Calcular subtotal recorriendo el JSON
    FOR v_detail IN SELECT * FROM jsonb_array_elements(p_details)
    LOOP
        v_qty := (v_detail->>'quantity')::INT;
        v_price := (v_detail->>'unit_price')::NUMERIC(10,2);
        v_subtotal := v_subtotal + (v_qty * v_price);
    END LOOP;

    v_tax := ROUND(v_subtotal * 0.18, 2);
    v_total := v_subtotal + v_tax;

    -- Insertar la orden
    INSERT INTO "order" (code, customer_id, subtotal, tax, total, status)
    VALUES (v_code, p_customer_id, v_subtotal, v_tax, v_total, 1)
    RETURNING order_id INTO v_order_id;

    -- Insertar detalles y descontar stock
    FOR v_detail IN SELECT * FROM jsonb_array_elements(p_details)
    LOOP
        v_prod_id := (v_detail->>'product_id')::INT;
        v_qty := (v_detail->>'quantity')::INT;
        v_price := (v_detail->>'unit_price')::NUMERIC(10,2);
        v_line_subtotal := v_qty * v_price;

        INSERT INTO order_detail (order_id, product_id, quantity, unit_price, subtotal)
        VALUES (v_order_id, v_prod_id, v_qty, v_price, v_line_subtotal);
```

```

        UPDATE product SET stock = stock - v_qty WHERE product_id = v_prod_id;
    END LOOP;

    RETURN v_order_id;
END;
$$ LANGUAGE plpgsql;

```

2) Invocación desde el proyecto web (EF Core + SqlQuery)

► Zoapex.Web/Data/OrderRepository.cs · RegisterOrderAsync()

```

// Procedimiento almacenado (función PostgreSQL): registro transaccional maestro-detalle
public async Task<int> RegisterOrderAsync(int? customerId, IReadOnlyList<CartLineDto> lines)
{
    if (lines.Count == 0)
        throw new InvalidOperationException("El carrito no puede estar vacío.");

    foreach (var line in lines)
    {
        if (line.Quantity <= 0)
            throw new InvalidOperationException("La cantidad debe ser mayor a cero.");

        var stock = await ctx.Products
            .AsNoTracking()
            .Where(p => p.ProductId == line.ProductId && p.Status == 1)
            .Select(p => p.Stock)
            .FirstOrDefaultAsync();

        if (stock < line.Quantity)
            throw new InvalidOperationException(
                $"Stock insuficiente para '{line.ProductName}'. Disponible: {stock}.");
    }

    var payload = lines.Select(l => new
    {
        product_id = l.ProductId,
        quantity = l.Quantity,
        unit_price = l.UnitPrice
    });

    var json = JsonSerializer.Serialize(payload);

    var orderId = await ctx.Database
        .SqlQuery<int>($"SELECT fn_register_order({customerId}, {json}::jsonb) AS \"Value\"")
        .SingleAsync();

    return orderId;
}

```

Explicación: la capa web serializa las líneas del carrito a JSON, invoca `fn_register_order` mediante `SqlQuery` y recibe el ID del pedido registrado. Si algo falla en la base de datos, no se confirma la venta.

D. AJAX - catálogo y carrito sin recargar

Justificación: al navegar el catálogo el cliente necesita filtrar productos y ver precio/stock al instante. En el carrito, al quitar un ítem debe actualizarse el resumen de totales sin recargar toda la página. Por eso implementé handlers Razor Pages que devuelven JSON y fetch en JavaScript.

1) Handler AJAX en el code-behind (Catalog.cshtml.cs)

► Zoapex.Web/Pages/Catalog.cshtml.cs · OnGetSearchAsync() / OnGetProductAsync()

```
// AJAX: filtra el catálogo sin recargar la página
public async Task<IActionResult> OnGetSearchAsync(string? q, int? categoryId)
{
    try
    {
        var products = await catalogRepo.GetCatalogAsync(q, categoryId);
        return new JsonResult(products);
    }
    catch (Exception ex)
    {
        return new JsonResult(new { error = true, message = ex.Message });
    }
}

// AJAX: devuelve precio y stock de un producto al seleccionarlo
public async Task<IActionResult> OnGetProductAsync(int id)
{
    try
    {
        var product = await catalogRepo.GetProductDetailAsync(id);
        if (product is null)
            return new JsonResult(new { found = false });

        return new JsonResult(new
        {
            found = true,
            product.ProductId,
            product.Code,
            product.Name,
            product.Price,
            product.Stock,
            product.CategoryName
        });
    }
    catch (Exception ex)
    {
        return new JsonResult(new { found = false, error = ex.Message });
    }
}
```

2) Fetch en la vista (Catalog.cshtml)

Explicación: al escribir en el buscador o cambiar la categoría, fetch llama al handler OnGetSearchAsync y actualiza solo la grilla de productos. Al pulsar 'Ver detalle', OnGetProductAsync devuelve el precio y stock sin recargar la página. En el carrito, refreshCart() usa ?handler=Summary para recalcular subtotal, IGV y total vía AJAX.

► Zoapex.Web/Pages/Catalog.cshtml · <script> loadProducts()

```
async function loadProducts() {
  const q = encodeURIComponent(searchInput.value.trim());
  const categoryId = categoryFilter.value;
  const url = `?handler=Search&q=${q}&categoryId=${categoryId}`;

  try {
    const response = await fetch(url);
    const products = await response.json();

    if (products.error) {
      grid.innerHTML = `<div class="empty-state">${products.message}</div>`;
      return;
    }

    if (!products.length) {
      grid.innerHTML = '<div class="empty-state">No se encontraron productos.</div>';
      return;
    }

    grid.innerHTML = products.map(p => `
      <article class="product-card">
        <span class="badge">${p.categoryName}</span>
        <p class="code">${p.code}</p>
        <h3>${p.name}</h3>
        <p class="desc">${p.description || 'Sin descripción'}</p>
        <p class="meta">$ / ${Number(p.price).toFixed(2)} · Stock: ${p.stock}</p>
        <button type="button" class="btn btn-outline btn-sm btn-select" data-id="${p.productId}">Ver detalle</button>
      </article>
    `).join("");

    grid.querySelectorAll('.btn-select').forEach(btn => {
      btn.addEventListener('click', () => showProductDetail(btn.dataset.id));
    });
  } catch {
    grid.innerHTML = '<div class="empty-state">Error al cargar productos. Revisa la conexión a la base de datos.</div>';
  }
}
```

E. Login de cliente

Complemento planificado en PA3: la web incluye registro e inicio de sesión de clientes. CustomerRepository valida credenciales con LINQ, PasswordHelper encripta la contraseña y CustomerSession guarda la sesión. El checkout del carrito requiere login y asocia el pedido al customer_id del cliente autenticado.



[Inicio](#) [Catálogo](#) [Carrito](#)

[Iniciar sesión](#) [Registrarse](#)



Bienvenido de vuelta

Inicia sesión para comprar y ver tus pedidos.

Correo electrónico

tu@correo.com

Contraseña

••••••••

Iniciar sesión

¿No tienes cuenta? [Regístrate aquí](#)

Cuenta demo

Correo: cliente@zoapex.com

Contraseña: zoapex123

► Zoapex.Web/Pages/Account/Login.cshtml

```
@page
@model Zoapex.Web.Pages.Account.LoginModel
@{
    ViewData["Title"] = "Iniciar sesión";
}

<section class="auth-card">
    <div class="auth-header">
        <span class="auth-icon"> </span>
        <h1>Bienvenido de vuelta</h1>
        <p>Inicia sesión para comprar y ver tus pedidos.</p>
    </div>

    @if (!string.IsNullOrEmpty(Model.ErrorMessage))
    {
        <div class="alert alert-error">@Model.ErrorMessage</div>
    }

    <form method="post" class="auth-form">
        <input type="hidden" name="returnUrl" value="@Model.ReturnUrl" />

        <label>
            Correo electrónico
            <input asp-for="Email" type="email" placeholder="tu@correo.com" required />
        </label>

        <label>
            Contraseña
            <input asp-for="Password" type="password" placeholder="••••••" required />
        </label>

        <button type="submit" class="btn btn-primary btn-block">Iniciar sesión</button>
    </form>

    <p class="auth-footer">
        ¿No tienes cuenta? <a asp-page="/Account/Register">Regístrate aquí</a>
    </p>

    <div class="demo-box">
        <strong>Cuenta demo</strong>
        <p>Correo: <code>cliente@zoapex.com</code></p>
        <p>Contraseña: <code>zoapex123</code></p>
    </div>
</section>
```

EVALUACIÓN FINAL

Cierra tu proyecto: Transaccionalidad Web + Reporte a Excel

Desarrollo de Aplicaciones Básico

Alumno: Stephano Camarena Villa

Proyecto: Zoapex | Tienda de mascotas

Fecha de entrega: 16 de julio de 2026

Esta sección cierra el ciclo del dato del proyecto Zoapex aplicando el flujo guardar → mostrar → exportar sobre la misma solución de la PA3/PA4. Se implementa una operación transaccional maestro-detalle en la web, un reporte de ventas con análisis y la descarga de ese reporte a Excel con EPPlus. Se respeta la regla de oro: la vista nunca consulta la base de datos directamente, siempre pasa por la capa de datos (repositorios y el servicio de exportación).

A. Escenario transaccional en la web

Qué se guarda: una Venta / Pedido con estructura cabecera-detalle: la tabla order (cabecera) y order_detail (varias líneas). La operación se graba 'todo o nada': si una línea falla, no se inserta nada. La cabecera, todos los detalles y el descuento de stock se ejecutan dentro de la misma transacción del procedimiento almacenado en PostgreSQL (fn_register_order). La vista (Pages/Cart.cshtml.cs) nunca toca la base de datos: llama a OrderRepository.RegisterOrderAsync, y ese método invoca la función fn_register_order.

► Zoapex.Web/Data/OrderRepository.cs · RegisterOrderAsync()

```
// Procedimiento almacenado (función PostgreSQL): registro transaccional maestro-detalle
public async Task<int> RegisterOrderAsync(int? customerId, IReadOnlyList<CartLineDto> lines)
{
    if (lines.Count == 0)
        throw new InvalidOperationException("El carrito no puede estar vacío.");

    foreach (var line in lines)
    {
        if (line.Quantity <= 0)
            throw new InvalidOperationException("La cantidad debe ser mayor a cero.");

        var stock = await ctx.Products
            .AsNoTracking()
            .Where(p => p.ProductId == line.ProductId && p.Status == 1)
            .Select(p => p.Stock)
            .FirstOrDefaultAsync();

        if (stock < line.Quantity)
            throw new InvalidOperationException(
                $"Stock insuficiente para '{line.ProductName}'. Disponible: {stock}.");
    }

    var payload = lines.Select(l => new
    {
        product_id = l.ProductId,
        quantity = l.Quantity,
        unit_price = l.UnitPrice
    });

    var json = JsonSerializer.Serialize(payload);

    var orderId = await ctx.Database
        .SqlQuery<int>($"SELECT fn_register_order({customerId}, {json}::jsonb) AS \"Value\"")
        .SingleAsync();

    return orderId;
}
```

► **zoapex-db/sql/zoapex_database.sql · fn_register_order()**

```
-- fn_register_order: Registra pedido + detalle en transacción y descuenta stock
-- p_customer_id puede ser NULL para ventas sin cliente registrado (mostrador)
CREATE OR REPLACE FUNCTION fn_register_order(
    p_customer_id INT,
    p_details JSONB -- [{product_id, quantity, unit_price}]
)
RETURNS INT AS $$
DECLARE
    v_order_id INT;
    v_next_id INT;
    v_code VARCHAR(12);
    v_subtotal NUMERIC(10,2) := 0;
    v_tax NUMERIC(10,2);
    v_total NUMERIC(10,2);
    v_detail JSONB;
    v_qty INT;
    v_price NUMERIC(10,2);
    v_prod_id INT;
    v_line_subtotal NUMERIC(10,2);
BEGIN
    -- Generar código del pedido
    SELECT COALESCE(MAX(order_id), 0) + 1 INTO v_next_id FROM "order";
    v_code := 'ORD' || LPAD(v_next_id::TEXT, 3, '0');

    -- Calcular subtotal recorriendo el JSON
    FOR v_detail IN SELECT * FROM jsonb_array_elements(p_details)
    LOOP
        v_qty := (v_detail->>'quantity')::INT;
        v_price := (v_detail->>'unit_price')::NUMERIC(10,2);
        v_subtotal := v_subtotal + (v_qty * v_price);
    END LOOP;

    v_tax := ROUND(v_subtotal * 0.18, 2);
    v_total := v_subtotal + v_tax;

    -- Insertar la orden
    INSERT INTO "order" (code, customer_id, subtotal, tax, total, status)
    VALUES (v_code, p_customer_id, v_subtotal, v_tax, v_total, 1)
    RETURNING order_id INTO v_order_id;

    -- Insertar detalles y descontar stock
    FOR v_detail IN SELECT * FROM jsonb_array_elements(p_details)
    LOOP
        v_prod_id := (v_detail->>'product_id')::INT;
        v_qty := (v_detail->>'quantity')::INT;
        v_price := (v_detail->>'unit_price')::NUMERIC(10,2);
        v_line_subtotal := v_qty * v_price;

        INSERT INTO order_detail (order_id, product_id, quantity, unit_price, subtotal)
        VALUES (v_order_id, v_prod_id, v_qty, v_price, v_line_subtotal);

        UPDATE product SET stock = stock - v_qty WHERE product_id = v_prod_id;
    END LOOP;

    RETURN v_order_id;
END;
$$ LANGUAGE plpgsql;
```

Mis pedidos

Hola María López, aquí está tu historial de compras.

¡Pedido registrado correctamente! Número de orden: 6

ORD006

Monday, 20 Jul 2026 · 19:44

Subtotal: S/ 51.40 · IGV: S/ 9.25 · **Total: S/ 60.65**

- Cat Kibble Bag 2kg — 1 × S/ 28.50 = S/ 28.50
- Retractable Leash 5m — 1 × S/ 22.90 = S/ 22.90

ORD001

Thursday, 02 Jul 2026 · 21:33

Subtotal: S/ 102.39 · IGV: S/ 18.43 · **Total: S/ 120.82**

- Cat Kibble Bag 2kg — 1 × S/ 28.50 = S/ 28.50
- Premium Dog Food 5kg — 1 × S/ 45.90 = S/ 45.90
- Rope Toy — 1 × S/ 12.99 = S/ 12.99
- Adjustable Collar — 1 × S/ 15.00 = S/ 15.00

☐	order_id i ▾	code varchar ▾	customer_id ▾	order_date timestamp ▾	subtotal num... ▾	tax numeric ▾	total numeric ▾	status int2 ▾
☐	5	ORD001	1	2026-07-03 02:33:46.948	102.39	18.43	120.82	1
☐	6	ORD006	1	2026-07-21 00:44:07.343	51.40	9.25	60.65	1

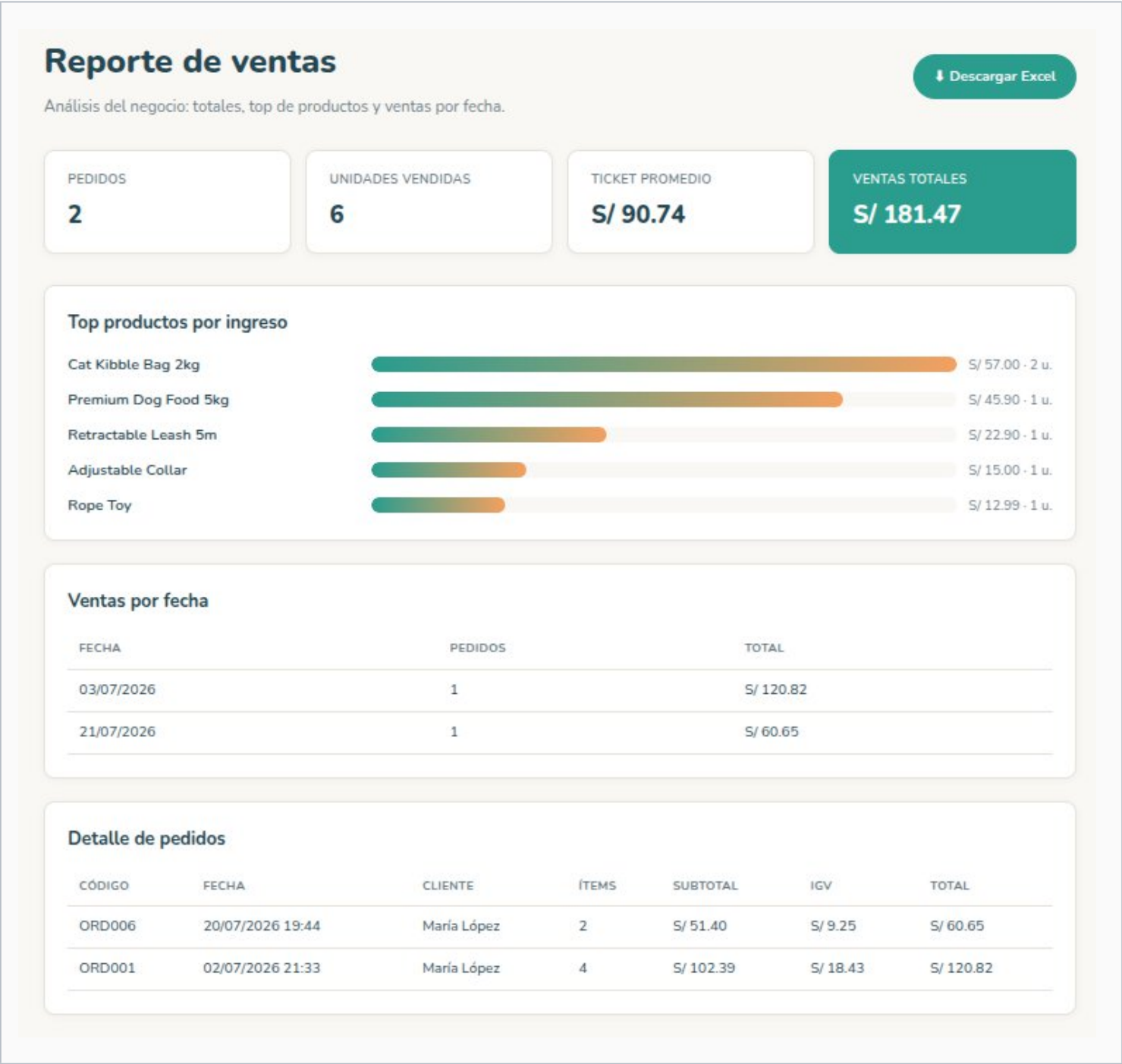
☐	detail_id i ▾	order_id i ▾	product_id ▾	quantity int4 ▾	unit_price numeric ▾	subtotal num... ▾
☐	9	5	2	1	28.50	28.50
☐	10	5	1	1	45.90	45.90
☐	11	5	3	1	12.99	12.99
☐	12	5	5	1	15.00	15.00
☐	13	6	2	1	28.50	28.50
☐	14	6	6	1	22.90	22.90

product ▾	code v... ▾	name varchar ▾	description varchar ▾	price n... ▾	stock i... ▾	min_stock ▾	category ▾	image_url ▾	status i... ▾	registered_at timestamp ▾
1	P001	Premium Dog Food 5kg	High-quality dry food for adult dogs	45.90	47	10	1	EMPTY	1	2026-05-25 04:33:34.144582
2	P002	Cat Kibble Bag 2kg	Balanced nutrition kibble for cats	28.50	28	5	1	EMPTY	1	2026-05-25 04:33:34.144582
3	P003	Rope Toy	Durable cotton rope toy for dogs	12.99	38	8	2	EMPTY	1	2026-05-25 04:33:34.144582
4	P004	Feather Wand	Interactive feather wand for cats	8.50	24	5	2	EMPTY	1	2026-05-25 04:33:34.144582
5	P005	Adjustable Collar	Nylon adjustable collar, size M	15.00	31	7	3	EMPTY	1	2026-05-25 04:33:34.144582
6	P006	Retractable Leash 5m	5-meter retractable leash for dogs	22.90	18	5	3	EMPTY	1	2026-05-25 04:33:34.144582
7	P007	Pet Shampoo 500ml	Gentle formula shampoo for dogs and cats	18.50	44	10	4	EMPTY	1	2026-05-25 04:33:34.144582
8	P008	Nail Clipper	Stainless steel nail clipper for pets	9.99	30	6	4	EMPTY	1	2026-05-25 04:33:34.144582

B. Reporte y descarga a Excel con EPPlus

B.1. El reporte en la web

Creé una página nueva /SalesReport ('Reporte de ventas') que presenta el dato como análisis, tabla y gráfico (no como texto crudo). Muestra cuatro indicadores (KPIs): número de pedidos, unidades vendidas, ticket promedio y ventas totales; un gráfico de barras con el Top 5 de productos por ingreso; una tabla de ventas por fecha; y una tabla con el detalle de pedidos.



B.2. Cómo llegué al reporte

Partí del dato que ya guardo en la Parte A: los pedidos (order) con sus líneas (order_detail). Me pregunté qué necesita un administrador para entender el negocio y decidí no mostrar los pedidos 'en crudo', sino resumirlos y analizarlos. Primero armé una sola consulta LINQ (GetSalesReportAsync) que trae los pedidos activos; a partir de esa información calculé los cuatro KPIs, siendo el ticket promedio las ventas totales divididas entre el número de pedidos. Luego quise responder '¿qué se vende más?', así que agrupé las líneas por producto y sumé sus ingresos para obtener el Top 5, que muestro como gráfico de barras (cada barra es proporcional al ingreso del producto respecto al mayor). Para ver la evolución en el tiempo, agrupé los mismos pedidos por fecha y sumé sus totales. Finalmente dejé una tabla con el detalle de pedidos para revisar caso por caso. Así el reporte pasa de 'texto crudo' a un tablero con análisis, gráfico y tablas, y ese mismo dato es el que después se descarga en Excel.

B.3. La descarga a Excel con EPPlus

El botón 'Descargar Excel' apunta al handler `OnGetExportAsync`, que genera y descarga un archivo .xlsx real. La respuesta usa el content-type de Office y un nombre de archivo con fecha y hora. El armado del libro (dos hojas: 'Resumen' con KPIs y gráfico, y 'Detalle de pedidos' con autofiltro y fila de totales) vive en `Data/SalesExcelExporter.cs`, y la licencia de uso académico se configura una sola vez en `Program.cs`.

► Zoapex.Web/Pages/SalesReport.cshtml.cs · OnGetExportAsync()

```
public async Task<IActionResult> OnGetExportAsync()
{
    var report = await orderRepo.GetSalesReportAsync();
    var bytes = exporter.Build(report);
    var fileName = $"reporte-ventas-{DateTime.Now:yyyyMMdd-HH:mm}.xlsx";

    return File(
        bytes,
        "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        fileName);
}
```

► Zoapex.Web/Data/SalesExcelExporter.cs · Build() / BuildSummarySheet()

```
public byte[] Build(SalesReportDto data)
{
    using var package = new ExcelPackage();

    BuildSummarySheet(package, data);
    BuildOrdersSheet(package, data);

    return package.GetAsByteArray();
}

private static void BuildSummarySheet(ExcelPackage package, SalesReportDto data)
{
    var ws = package.Workbook.Worksheets.Add("Resumen");

    ws.Cells["A1"].Value = "Zoapex — Reporte de ventas";
    ws.Cells["A1:D1"].Merge = true;
    StyleTitle(ws.Cells["A1"]);

    ws.Cells["A2"].Value = $"Generado: {DateTime.Now:dd/MM/yyyy HH:mm}";
    ws.Cells["A2:D2"].Merge = true;

    // KPIs
    var kpis = new (string Label, object Value, bool IsMoney)[]
    {
        ("Pedidos", data.OrderCount, false),
        ("Unidades vendidas", data.UnitsSold, false),
        ("Ticket promedio", data.AverageTicket, true),
        ("Ventas totales", data.TotalRevenue, true),
    };

    var row = 4;
    foreach (var (label, value, isMoney) in kpis)
    {
        ws.Cells[row, 1].Value = label;
        ws.Cells[row, 1].Style.Font.Bold = true;
        ws.Cells[row, 2].Value = value;
        if (isMoney)
```

```

        ws.Cells[row, 2].Style.Numberformat.Format = Money;
        row++;
    }

    // Top productos
    var top = row + 1;
    ws.Cells[top, 1].Value = "Top productos";
    ws.Cells[top, 1, top, 3].Merge = true;
    StyleHeader(ws.Cells[top, 1, top, 3]);

    var head = top + 1;
    ws.Cells[head, 1].Value = "Producto";
    ws.Cells[head, 2].Value = "Cantidad";
    ws.Cells[head, 3].Value = "Ingreso";
    StyleHeader(ws.Cells[head, 1, head, 3]);

    var r = head + 1;
    foreach (var p in data.TopProducts)
    {
        ws.Cells[r, 1].Value = p.ProductName;
        ws.Cells[r, 2].Value = p.Quantity;
        ws.Cells[r, 3].Value = p.Revenue;
        ws.Cells[r, 3].Style.Numberformat.Format = Money;
        r++;
    }

    // Gráfico de barras de top productos (evidencia visual dentro del Excel)
    if (data.TopProducts.Count > 0)
    {
        var chart = ws.Drawings.AddBarChart("TopProductsChart", eBarChartType.BarClustered);
        chart.Title.Text = "Top productos por ingreso";
        chart.SetPosition(head - 1, 0, 4, 0);
        chart.SetSize(460, 260);
        var series = chart.Series.Add(ws.Cells[head + 1, 3, r - 1, 3], ws.Cells[head + 1, 1, r - 1, 1]);
        series.Header = "Ingreso";
        chart.Legend.Remove();
    }

    ws.Cells[ws.Dimension.Address].AutoFitColumns();
}

```

	A	B	C	D	E	F	G	H	I	J	K
1	Zoapex — Reporte de ventas										
2	Generado: 20/07/2026 20:02										
3											
4	Pedidos	2									
5	Unidades vendidas	6									
6	Ticket promedio	S/ 90.74									
7	Ventas totales	S/ 181.47									
8											
9	Top productos										
10	Producto	Cantidad	Ingreso								
11	Cat Kibble Bag 2kg	2	S/ 57.00								
12	Premium Dog Food 5kg	1	S/ 45.90								
13	Retractable Leash 5m	1	S/ 22.90								
14	Adjustable Collar	1	S/ 15.00								
15	Rope Toy	1	S/ 12.99								
16											
17											
18											
19											
20											
21											
22											
23											

Top productos por Ingreso

Producto	Ingreso (S/)
Cat Kibble Bag 2kg	57.00
Premium Dog Food 5kg	45.90
Retractable Leash 5m	22.90
Adjustable Collar	15.00
Rope Toy	12.99

	A	B	C	D	E	F	G
1	Código	Fecha	Cliente	Ítems	Subtotal	IGV	Total
2	ORD006	21/07/2026 00:44	María López	2	S/ 51.40	S/ 9.25	S/ 60.65
3	ORD001	03/07/2026 02:33	María López	4	S/ 102.39	S/ 18.43	S/ 120.82
4	TOTAL				S/ 153.79	S/ 27.68	S/ 181.47

B.4. La query adjunta (LINQ que alimenta el reporte)

Consulta LINQ de agregación en la capa de datos (Data/OrderRepository.cs, método GetSalesReportAsync). Trae los pedidos con EF Core y calcula los KPIs, el top de productos (agrupando por producto y sumando cantidad e ingreso) y las ventas por fecha.

► Zoapex.Web/Data/OrderRepository.cs · GetSalesReportAsync()

```
// LINQ: reporte de ventas con análisis (KPIs, top productos, ventas por fecha, detalle)
// Es la "query adjunta" que alimenta el reporte web y la descarga a Excel.
public async Task<SalesReportDto> GetSalesReportAsync()
{
    // Filas de pedidos (cabecera + nombre de cliente + n° de ítems)
    var orders = await ctx.Orders
        .AsNoTracking()
        .Where(o => o.Status == 1)
        .OrderByDescending(o => o.OrderDate)
        .Select(o => new SalesOrderRowDto(
            o.Code,
            o.OrderDate,
            o.Customer != null ? o.Customer.FirstName + " " + o.Customer.LastName : "Mostrador",
            o.Details.Sum(d => (int?)d.Quantity) ?? 0,
            o.Subtotal,
            o.Tax,
            o.Total))
        .ToListAsync();

    // Detalle plano (traducible a SQL) y agrupación en memoria por producto
    var detailRows = await ctx.OrderDetails
        .AsNoTracking()
        .Where(d => d.Order.Status == 1)
        .Select(d => new { d.Product.Name, d.Quantity, d.Subtotal })
        .ToListAsync();

    var topProducts = detailRows
        .GroupBy(d => d.Name)
        .Select(g => new TopProductDto(
            g.Key,
            g.Sum(d => d.Quantity),
            g.Sum(d => d.Subtotal)))
        .OrderByDescending(t => t.Revenue)
        .Take(5)
        .ToList();

    // Ventas por fecha, calculadas desde los pedidos ya materializados
    var salesByDate = orders
        .GroupBy(o => o.OrderDate.Date)
        .Select(g => new SalesByDateDto(
            g.Key,
            g.Count(),
            g.Sum(o => o.Total)))
        .OrderBy(s => s.Date)
        .ToList();

    var orderCount = orders.Count();
    var totalRevenue = orders.Sum(o => o.Total);
    var unitsSold = orders.Sum(o => o.Items);
    var averageTicket = orderCount > 0 ? Math.Round(totalRevenue / orderCount, 2) : 0m;

    return new SalesReportDto(
        orderCount,
```

```

totalRevenue,
averageTicket,
unitsSold,
topProducts,
salesByDate,
orders);
}

```

C. Seguridad (opcional — puntos extra)

La seguridad se implementó con el enfoque actual de ASP.NET Core Razor Pages: autenticación por cookie, autorización por políticas y roles, y convenciones de acceso por página (sin el antiguo proveedor Membership ni reglas por Web.config). Los cuatro puntos de la Parte C están cubiertos:

- Roles por claims: el usuario se autentica con `SignInAsync` y su identidad se firma en una cookie cifrada; el nombre y el rol (Admin / Cliente, tomado de la columna 'role' de customer) viajan como claims. La lógica está en `Data/CustomerIdentity.cs`.
- Reglas de acceso a las páginas del sitio: se centralizan en `Program.cs` con convenciones y políticas — `AuthorizePage("/SalesReport", "EsAdmin")` con política `RequireRole("Admin")`, y `AuthorizePage("/Orders")` que exige autenticación. En el menú (`_Layout`) el enlace 'Reporte' solo aparece con `User.IsInRole("Admin")`.
- Formulario de Login (`/Account/Login`): valida las credenciales contra el hash de contraseña (`PasswordHelper` con `PasswordHasher`) y, si son correctas, firma la cookie con los claims; el Logout borra la cookie con `SignOutAsync`.
- Caducidad de la sesión: parte servidor — la cookie expira con `Expires(TimeSpan (30 min))` y `SlidingExpiration` (se renueva con la actividad); parte cliente — un JavaScript en el `_Layout` detecta la inactividad, avisa y luego redirige a `/Account/Logout` para cerrar la sesión. Los minutos de aviso y de cierre se configuran en `appsettings.json` (sección Security).

Cuentas demo — Admin: `admin@zoapex.com / zoapex123` · Cliente: `cliente@zoapex.com / zoapex123`.

► Zoapex.Web/Program.cs · autenticación por cookie + políticas de rol

```

// Autenticación por cookie (reemplaza al esquema basado en ISession)
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Account/Login";
        options.LogoutPath = "/Account/Logout";
        options.AccessDeniedPath = "/Account/Login";
        options.ExpireTimeSpan = TimeSpan.FromMinutes(timeoutMin);
        options.SlidingExpiration = true; // la cookie se renueva con la actividad
        options.Cookie.HttpOnly = true;
        options.Cookie.IsEssential = true;
    });

// Políticas de autorización por rol (reemplazan al roleManager del stack antiguo)
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("EsAdmin", policy => policy.RequireRole("Admin"));
});

// Reglas de acceso a las páginas del sitio (convenciones de Razor Pages).
// El resto de páginas queda anónimo (catálogo y carrito son públicos).
builder.Services.AddRazorPages(options =>
{
    options.Conventions.AuthorizePage("/Orders"); // requiere haber iniciado sesión
    options.Conventions.AuthorizePage("/SalesReport", "EsAdmin"); // solo administrador
});

```


► Zoapex.Web/Pages/Account/Login.cshtml.cs · OnPostAsync()

```
public async Task<IActionResult> OnPostAsync(string? returnUrl)
{
    returnUrl = SafeReturnUrl(returnUrl);

    var customer = await customerRepo.ValidateLoginAsync(Email, Password);
    if (customer is null)
    {
        ErrorMessage = "Correo o contraseña incorrectos.";
        return Page();
    }

    // Firma la cookie de autenticación con el nombre, correo, rol e Id del cliente
    await HttpContext.SignInAsync(
        CookieAuthenticationDefaults.AuthenticationScheme,
        CustomerIdentity.Build(customer),
        new AuthenticationProperties { IsPersistent = false });

    return LocalRedirect(ReturnUrl);
}
```

[Inicio](#)[Catálogo](#)[Carrito](#)[Mis pedidos](#)[Reporte](#)

Hola, Admin Zoapex

[Salir](#)